

```

class QuickSort:

    def __init__(self):

        self.comparisons = 0

        self.swaps = 0

        self.recursions = 0


    def partition(self, arr, low, high):

        pivot = arr[high]    # Choose the last element as pivot

        i = low - 1

        for j in range(low, high):

            self.comparisons += 1 # Count comparisons

            if arr[j] < pivot:

                i += 1

                arr[i], arr[j] = arr[j], arr[i] # Swap smaller elements

                self.swaps += 1

        # Place pivot in correct position

        arr[i + 1], arr[high] = arr[high], arr[i + 1]

        self.swaps += 1

        return i + 1


    def quick_sort(self, arr, low, high):

        if low < high:

            self.recursions += 1

```

```
pi = self.partition(arr, low, high) # Partition index
self.quick_sort(arr, low, pi - 1) # Sort left half
self.quick_sort(arr, pi + 1, high) # Sort right half
```

```
def sort_and_analyze(self, arr):
    # Reset counters
    self.comparisons = 0
    self.swaps = 0
    self.recursions = 0

    self.quick_sort(arr, 0, len(arr) - 1)

    return arr, self.comparisons, self.swaps, self.recursions
```

```
# Test Data
```

```
unsorted_list = [12, 20, 22, 16, 25, 18, 8, 10, 6, 15]
```

```
sorted_list = [6, 8, 10, 12, 15, 16, 18, 20, 22, 25]
```

```
# For Unsorted List
```

```
qs1 = QuickSort()
```

```
sorted_arr1, comp1, swaps1, rec1 =
```

```
qs1.sort_and_analyze(unsorted_list.copy())
```

```
# For Already Sorted List
```

```
qs2 = QuickSort()
```

```
sorted_arr2, comp2, swaps2, rec2 =
```

```
qs2.sort_and_analyze(sorted_list.copy())
```

```
# Print Results
```

```
print("Unsorted List Performance:")
```

```
print("Sorted Array:", sorted_arr1)
```

```
print(f"Comparisons: {comp1}, Swaps: {swaps1}, Recursive Calls: {rec1}")
```

```
print("\nAlready Sorted List Performance:")
```

```
print("Sorted Array:", sorted_arr2)
```

```
print(f"Comparisons: {comp2}, Swaps: {swaps2}, Recursive Calls: {rec2}")
```

```
import heapq
```

```
class HuffmanNode:
```

```
    def __init__(self, char, freq, order):
```

```
        self.char = char
```

```
        self.freq = freq
```

```
        self.left = None
```

```
        self.right = None
```

```
        self.order = order # tie-breaker to keep insertion order
```

```
    def __lt__(self, other):
```

```
        # Compare by frequency first, then by order
```

```
        if self.freq == other.freq:
```

```
            return self.order < other.order
```

```
        return self.freq < other.freq
```

```
def build_huffman_tree(frequencies):
```

```
    # Sort items alphabetically to fix order
```

```
    heap = []
```

```
    for order, (char, freq) in enumerate(sorted(frequencies.items())):
```

```
        heapq.heappush(heap, HuffmanNode(char, freq, order))
```

```
    while len(heap) > 1:
```

```
        left = heapq.heappop(heap)
```

```
        right = heapq.heappop(heap)
```

```

        merged = HuffmanNode(None, left.freq + right.freq, left.order)
        merged.left = left
        merged.right = right
        heapq.heappush(heap, merged)

    return heap[0]

def generate_codes(node, code="", codes={}):
    if node is None:
        return

    if node.char is not None:
        codes[node.char] = code
        return

    generate_codes(node.left, code + "0", codes)
    generate_codes(node.right, code + "1", codes)
    return codes

# Given frequencies
frequencies = {'A': 15, 'B': 25, 'C': 5, 'D': 7, 'E': 10, 'F': 13, 'G': 9}

# Step 1: Build Huffman Tree
root = build_huffman_tree(frequencies)

# Step 2: Generate Huffman Codes
codes = generate_codes(root)

print("Huffman Codes for given characters:")
for ch in sorted(codes.keys()):

```

```
print(f"{ch}: {codes[ch]}")
```

```
# Step 3: Calculate average bits
```

```
total_bits = sum(len(codes[ch]) * frequencies[ch] for ch in frequencies)
```

```
avg_bits = total_bits / sum(frequencies.values())
```

```
print("\nAverage bits required per character:", round(avg_bits, 2))
```

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-
scale=1.0">

<title>Patient Satisfaction Survey</title>


<style>

body{

    font-family: Verdana, sans-serif;

    background-color: #e8f0fe;

    margin: 0;

    padding: 0;

}

.container {

    background-color: #fff;

    width: 45%;

    margin: 50px auto;

    padding: 25px;

    border-radius: 10px;

    box-shadow: 0px 0px 8px rgba(0,0,0,0.3);

}

h2{

    text-align: center;

    color: #333;
```

```
}  
  
label {  
    font-weight: bold;  
    display: block;  
    margin-top: 10px;  
    text-align: left;  
}  
  
input[type="text"], select, textarea {  
    width: 100%;  
    padding: 8px;  
    margin-top: 5px;  
    border: 1px solid #ccc;  
    border-radius: 5px;  
}  
  
.radio-group {  
    margin-top: 5px;  
    display: flex;  
    justify-content: space-around;  
}  
  
button {  
    padding: 10px 20px;  
    margin: 10px 5px;  
    border: none;  
    color: white;  
    border-radius: 5px;  
    cursor: pointer;  
}  
  
.submit-btn { background-color: #28a745; }
```



```
.reset-btn { background-color: #dc3545; }

</style>

<script>

function validateSurvey() {

    const name = document.forms["surveyForm"]["name"].value.trim();

    const fileNumber =
document.forms["surveyForm"]["fileNumber"].value.trim();

    const unit = document.forms["surveyForm"]["unit"].value;

    const treatment =
document.querySelector('input[name="treatment"]:checked');

    const facilities =
document.querySelector('input[name="facilities"]:checked');

    if (name === "" || fileNumber === "" || unit === "" || !treatment ||
!facilities) {

        alert("Please fill in all required fields!");

        return false;

    }

    return true;
}

</script>

</head>

<body>

<div class="container">

    <h2>Patient Satisfaction Survey</h2>

    <form name="surveyForm" action="storeSurvey.jsp" method="post"
onsubmit="return validateSurvey();">
```

<label>Patient's Name:</label>

<input type="text" name="name" placeholder="Enter patient's name">

<label>Patient's File Number:</label>

<input type="text" name="fileNumber" placeholder="Enter hospital file number">

<label>Hospital Unit Admitted:</label>

<select name="unit">

<option value="">--Select Unit--</option>

<option value="Surgery">Surgery</option>

<option value="Medicine">Medicine</option>

<option value="Pediatrics">Pediatrics</option>

<option value="Orthopedics">Orthopedics</option>

</select>

<label>Overall Treatment Satisfaction:</label>

<div class="radio-group">

<label><input type="radio" name="treatment" value="Very Satisfied"> Very Satisfied</label>

<label><input type="radio" name="treatment" value="Satisfied"> Satisfied</label>

<label><input type="radio" name="treatment" value="Not Satisfied"> Not Satisfied</label>

</div>

<label>Medical Facilities Satisfaction:</label>

```
<div class="radio-group">

    <label><input type="radio" name="facilities" value="Very
Satisfied"> Very Satisfied</label>

    <label><input type="radio" name="facilities" value="Satisfied">
Satisfied</label>

    <label><input type="radio" name="facilities" value="Not Satisfied">
Not Satisfied</label>

</div>
```

```
<label>Overall Comments:</label>

<textarea name="comments" rows="4" placeholder="Write your
comments here..."></textarea>
```

```
<div style="text-align:center;">

    <button type="submit" class="submit-btn">Submit</button>

    <button type="reset" class="reset-btn">Reset</button>

</div>
```

```
</form>
```

```
</div>
```

```
</body>
```

```
</html>
```

```
<%@ page import="java.sql.*" %>

<%

String name = request.getParameter("name");
String fileNumber = request.getParameter("fileNumber");
String unit = request.getParameter("unit");
String treatment = request.getParameter("treatment");
String facilities = request.getParameter("facilities");
String comments = request.getParameter("comments");

if(name == null || name.trim().isEmpty() ||
    fileNumber == null || fileNumber.trim().isEmpty() ||
    unit == null || unit.trim().isEmpty() ||
    treatment == null || facilities == null) {

    out.println("<script>alert('All fields are required!');
window.history.back();</script>");
} else {

    Connection conn = null;
    PreparedStatement stmt = null;

    try {

        Class.forName("com.mysql.cj.jdbc.Driver");
```

```
        conn =  
DriverManager.getConnection("jdbc:mysql://localhost:3306/hospital_db"  
, "root", "password");
```

```
        String sql = "INSERT INTO patient_survey (name, file_number, unit,  
treatment, facilities, comments) VALUES (?, ?, ?, ?, ?, ?)";
```

```
        stmt = conn.prepareStatement(sql);
```

```
        stmt.setString(1, name);
```

```
        stmt.setString(2, fileNumber);
```

```
        stmt.setString(3, unit);
```

```
        stmt.setString(4, treatment);
```

```
        stmt.setString(5, facilities);
```

```
        stmt.setString(6, comments);
```

```
        int inserted = stmt.executeUpdate();
```

```
        if (inserted > 0) {
```

```
            out.println("<script>alert('Survey submitted successfully!');  
window.location.href='patientSurvey.html';</script>");
```

```
        }
```

```
    } catch (Exception e) {
```

```
        out.println("<script>alert('Database Error: " + e.getMessage() +  
"');</script>");
```

```
    } finally {
```

```
        if (stmt != null) stmt.close();
```

```
        if (conn != null) conn.close();
```

```
    }
```

```
}
```

```
%>
```

```
CREATE DATABASE hospital_db;
```

```
USE hospital_db;
```

```
CREATE TABLE patient_survey (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    file_number VARCHAR(50) NOT NULL,  
    unit VARCHAR(50) NOT NULL,  
    treatment VARCHAR(50) NOT NULL,  
    facilities VARCHAR(50) NOT NULL,  
    comments TEXT  
);
```

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-
scale=1.0">

<title>Cookie Manager</title>


<style>

  body {

    font-family: Verdana, sans-serif;

    background-color: #eef3f7;

    text-align: center;

    margin-top: 60px;

  }


  h2 {

    color: #333;

    margin-bottom: 25px;

  }


  button {

    padding: 10px 20px;

    margin: 6px;

    border: none;

    border-radius: 6px;
```

```
background-color: #2196F3;

color: white;

cursor: pointer;

transition: 0.3s;

}


button:hover {

background-color: #1976D2;

}


.btn-group {

margin-bottom: 15px;

}

</style>


<script>

// Set cookie function

function setCookie(name, value, days) {

const date = new Date();

date.setTime(date.getTime() + (days * 24 * 60 * 60 * 1000));

document.cookie = `${name}=${value};

expires=${date.toUTCString()}; path=/`;

alert(`${name} has been set successfully!`);

}


// Get cookie function

function getCookie(name) {

const cookieArr = document.cookie.split("; ");
```



```

    for (let c of cookieArr) {
        const [key, val] = c.split("=");
        if (key === name) {
            alert(` ${name} Value: ${val} `);
            return;
        }
    }
    alert(` ${name} not found! `);
}

// Delete cookie function
function deleteCookie(name) {
    document.cookie = ` ${name}=; expires=Thu, 01 Jan 1970 00:00:00
UTC; path=/; `;
    alert(` ${name} has been deleted. `);
}

// Display all cookies
function showAllCookies() {
    const allCookies = document.cookie;
    if (allCookies) {
        alert(` Stored Cookies: ${allCookies} `);
    } else {
        alert("No cookies available!");
    }
}
</script>
</head>

```

```
<body>

  <h2>Cookie Manager</h2>

  <div class="btn-group">

    <button onclick="setCookie('cookie1', 'ValueOfCookie1', 7)">Set
Cookie1</button>

    <button onclick="getCookie('cookie1')">Get Cookie1</button>

    <button onclick="deleteCookie('cookie1')">Delete Cookie1</button>

  </div>

  <div class="btn-group">

    <button onclick="setCookie('cookie2', 'ValueOfCookie2', 7)">Set
Cookie2</button>

    <button onclick="getCookie('cookie2')">Get Cookie2</button>

    <button onclick="deleteCookie('cookie2')">Delete Cookie2</button>

  </div>

  <button onclick="showAllCookies()">Display All Cookies</button>

</body>

</html>
```